PART 2 · 2H

Claude Code로 백지에서 저장소 만들기

설치 · 로그인 · 뼈대 · 커밋 · 푸시

Claude Code로 만드는 나만의 GitHub 사이트

사람은 가입과 로그인 승인만 하고, 나머지는 Claude Code에 한국어로 지시한다.
올리기 전에 비밀값을 걸러내는 습관을 실제 사고 사례로 익힌다.

이번 파트에서 가져갈 세 가지

실습의 원칙 — 무엇을 직접 하고 무엇을 맡기는가

1

사람이 직접 할 일은 가입과 로그인 승인뿐이다

계정 생성과 브라우저 승인은 본인 인증이라 대신할 수 없다. 나머지는 지시로 해결한다.

2

지시는 한 번에 한 단계씩 한다

'뼈대 만들기 → 자료 넣기 → 커밋·푸시'처럼 끊어 지시하면 결과를 확인하며 나아갈 수 있다.

3

올리기 전에 비밀값을 걸러낸다

한 번 공개 저장소에 올라간 키는 지워도 이미 복제됐다고 보고 폐기·재발급해야 한다.

흐름: 0 준비 → 1 뼈대 → 2 자료 → 3 커밋·푸시 → 5 반복 (4단계 배포는 Part 3)

Claude Code는 터미널에서 일하는 AI 에이전트다

기본 지식 · 터미널(terminal)과 에이전트(agent)의 뜻

터미널

- 글자로 컴퓨터에 지시하는 창
- macOS: 터미널 앱 / Windows: PowerShell
- 검은 화면이지만 겁낼 필요는 없다

Claude Code

- 터미널 안에서 한국어 지시를 받는다
- 파일 생성·명령 실행·커밋을 대신한다
- 실행 전에 승인을 요청한다

사용 조건: Claude Pro·Max·Team·Enterprise 구독 (무료 플랜은 Claude Code 미포함)

본인 인증만 사람이 하고 나머지는 지시로 해결한다

역할 분담 · 이 표를 기억하면 막혔을 때 누가 할 일인지 바로 안다

사람이 직접 (3가지)

- GitHub 회원가입 (아이디 = 사이트 주소)
- 로그인 승인: 브라우저에서 Authorize
- 올릴 자료와 올리지 않을 자료 고르기

Claude Code에 지시 (나머지)

- Git·gh 설치 여부 확인과 설치
- 폴더·README·.gitignore 만들기
- 커밋·저장소 생성·푸시
- 오류 메시지 해석과 해결

설치는 운영체제별 명령 한 줄로 끝난다

0단계 준비 ① · 공식 네이티브 설치 (Node.js 불필요, 자동 업데이트)

[입력] 명령어 · macOS 터미널

```
curl -fsSL https://claude.ai/install.sh | bash
```

[입력] 명령어 · Windows PowerShell

```
irm https://claude.ai/install.ps1 | iex
```

[입력] 명령어 · 설치 확인

```
claude --version
```

[입력] 명령어 · 작업 폴더에서 시작

```
cd my-ai-course  →  claude
```

막히면

- claude를 찾을 수 없다고 나오면 터미널 창을 새로 연다
- Windows는 Git for Windows 설치를 권장한다 (Claude Code가 Git Bash 사용)

GitHub 로그인은 본인이 브라우저에서 승인한다

0단계 준비 ② · 앞에 붙는 ! 는 'Claude가 아니라 내가 직접 실행'이라는 뜻이다

[입력] 프롬프트 · Claude Code

```
Git과 GitHub CLI(gh)가 설치돼 있는지 확인하고, 없으면 설치해줘.
```

[입력] 명령어 · Claude Code 입력창에 직접 입력

```
! gh auth login
```

화면에 나오는 선택지

- GitHub.com → HTTPS → Login with a web browser → 표시된 8자리 코드를 브라우저에 입력

첫 지시 한 문장으로 저장소의 뼈대가 생긴다

1단계 뼈대 · 폴더 구조는 강사 저장소(Part 1)를 본뜬다

[입력] 프롬프트 · Claude Code

```
이 폴더를 Git 저장소로 만들어줘.
Day1~Day6, data 폴더를 만들고,
과정 소개와 폴더 표가 들어간
README.md, 그리고 .gitignore도
만들어줘.
```

[출력] Claude가 생성 · 폴더 구조

```
my-ai-course/
 ├── .git/           (기록 보관소)
 ├── Day1/ ... Day6/
 ├── data/
 ├── README.md       (과정 소개·폴더 표)
 └── .gitignore      (올리지 않을 목록)
```

확인

- README.md를 열어 과정 소개가 맞는지 읽어 본다

자료를 넣은 뒤 README를 실제 파일에 맞춘다

2단계 자료 · 파일 복사는 직접, 정리는 Claude Code가 한다

[입력] 프롬프트 · Claude Code

```
README의 폴더 표를 실제 파일에 맞게 고쳐줘. 파일마다 한 줄 설명도 붙여줘.
```

넣을 자료

- 실습 결과물 PDF·HTML
- 공개해도 되는 데이터 (출처 표기)
- 발표 자료는 PDF로 변환해 넣기

넣지 않을 자료

- 회사 내부 문서·고객 정보
- API 키·비밀번호가 든 파일
- 100MB가 넘는 영상·원본 파일

커밋·저장소 생성·푸시를 한 번에 지시한다

3단계 업로드 · gh가 GitHub에 저장소를 만들고 기록을 올린다

[입력] 프롬프트 · Claude Code

```
지금까지 작업을 커밋하고, gh로 public 저장소 my-ai-course를 만들어서 푸시해줘.
```

[출력] 결과 · 터미널 메시지 (예시)

```
✓ Created repository 아이디/my-ai-course on GitHub
✓ Pushed commits to https://github.com/아이디/my-ai-course
```

확인

- 브라우저에서 github.com/아이디/my-ai-course 를 열어 README가 보이면 성공

이후 수정은 같은 문장 하나로 반복한다

5단계 반복 · 자료를 고치거나 추가할 때마다

[입력] 프롬프트 · Claude Code

바뀐 내용 커밋하고 푸시해줘. 커밋 메시지는 무엇을 왜 바꿨는지 한국어로 써줘.

좋은 커밋 메시지

- Day2 크롤링 실습 결과 PDF 추가
- README 폴더 표에 data 출처 추가

나쁜 커밋 메시지

- 수정
- update
- asdf

.gitignore는 올라가면 안 되는 파일을 미리 막는다

기본 지식 · 목록에 적힌 파일은 커밋 대상에서 자동으로 빠진다

.gitignore 항목	무엇인가	막는 이유
.env	API 키·비밀번호를 적어 두는 파일	비밀값 유출
.DS_Store · Thumbs.db	운영체제가 만드는 숨김 파일	불필요한 잡음
node_modules/	설치된 프로그램 묶음	용량이 크고 다시 설치 가능
*.local	내 컴퓨터 전용 설정	개인 환경 노출

지시 예: ".gitignore에 .env와 운영체제 숨김 파일을 추가해줘"

공개 저장소에 올라간 키 하나가 5년 동안 노출됐다

사례 · Toyota T-Connect, 2022년 10월 공개 (TechCrunch)

2017.12

협력사가 소스코드를
공개 저장소에 올림

약 5년

2022.9 발견까지
아무도 몰랐다

296,019

접근 가능했던
고객 이메일 수

무슨 일이 있었나

- 코드 안에 서버 접근 키가 들어 있었다
- 열람 여부를 확인할 수 없었다

교훈

- 키는 코드가 아니라 .env에 둔다
- 새면 지우지 말고 폐기·재발급한다

공개 GitHub에 새는 비밀값은 해마다 늘고 있다

사례 · GitGuardian, State of Secrets Sprawl 2026 (2025년 집계)

약 2,900만

2025년 공개 커밋에서
새로 발견된 비밀값

+34%

전년 대비 증가
(역대 최대 폭)

128만

AI 서비스 키
(전년 대비 +81%)

64%

2022년 유출 키 중
4년 뒤에도 유효

AI 실습에서 특히 조심할 것

- Claude·OpenAI·Gemini API 키를 코드에 직접 붙여넣지 않는다
- 지시 예: "커밋하기 전에 비밀 키처럼 보이는 문자열이 없는지 검사해줘"

큰 파일은 GitHub가 경고하거나 막는다

제약 · GitHub Docs 'About large files on GitHub' 기준

기준	GitHub의 동작	대응
50MiB 초과	경고 (푸시는 됨)	압축하거나 PDF로 변환
100MiB 초과	푸시 차단	빼거나 Git LFS 사용
웹 화면 업로드	파일당 25MiB까지	Claude Code로 푸시
저장소 전체	1GB 이하 권장	영상은 별도 저장소·드라이브

MiB ≈ MB. 강의 덱은 원본 PPTX 대신 PDF로 올리면 용량 문제가 대부분 사라진다

실습 결과와 습관을 함께 점검한다

점검 · 내 저장소 주소를 제출하고 아래 질문에 답한다

Q1	내 저장소 주소와 README 첫 문장을 제출하라	**결과**
Q2	.gitignore에 무엇을 넣었고 왜 넣었는가	**보안**
Q3	API 키를 공개 저장소에 올렸다면 가장 먼저 무엇을 하는가	**판단**
Q4	! gh auth login 은 왜 직접 실행해야 하는가	**개념**

저장소는 생겼다, 이제 사이트로 공개한다

종합 · Part 3에서 같은 저장소를 웹사이트로 배포한다

1

다섯 문장이면 저장소가 완성된다

설치 확인 → 뼈대 → README 정리 → 커밋·푸시 → 이후 반복.

2

본인 인증은 직접, 나머지는 지시

막히면 '누가 할 일인가'부터 따진다. 오류 메시지는 그대로 Claude Code에 붙여넣는다.

3

새면 지우지 말고 폐기한다

Toyota는 5년, 유출 키의 64%는 4년 뒤에도 유효했다. 올리기 전에 거른다.

다음 · Part 3 배포의 이해와 GitHub Pages